FraudShield AI — Learn and present your project

Project: Credit Card Fraud Detection Using Machine Learning

Application: FraudShield AI

Website: https://creditvault.streamlit.app/

This guide describes the current FraudShield application. Older ULB, PaySim, and Sparkov dashboards in the repository are historical experiments.

1. Your project in one minute

“FraudShield AI is a credit-card fraud detection project. It uses Sparkov, a publicly available synthetic credit-card transaction dataset. I compare Logistic Regression and XGBoost using a chronological training, validation, and test split. The application accepts readable payment details and spending context, applies the saved preprocessing and selected model, and returns a predicted class and review score. It also supports batch analysis, measured model results, SHAP explanations, and session monitoring. The data is synthetic, so the project demonstrates the complete ML workflow rather than proving real-bank performance.”

That is a truthful, complete project description. You do not need to describe yourself as an ML expert.

2. What a dataset is

A dataset is a table of examples. Here, a row represents a card purchase. Columns describe the amount, date, merchant, category, customer, and the recorded answer.

The answer is called a **label**: 0 means legitimate and 1 means fraud. The model sees labels while learning and while being evaluated. It never receives the label as an input when making a prediction.

The selected source contains 555,719 synthetic transactions across 924 synthetic accounts. There are 2,145 recorded fraud cases. Amounts are in US dollars, abbreviated USD.

Synthetic means the records were generated by a simulator. They are not actual customers' banking records. The model training and calculations are still genuine, but the measured results describe this simulated dataset.

We used the publisher's fraudTest.csv portion and divided it into this project's own training, validation, and test periods. Its original filename does not mean that all of it is our final test set.

3. Why we selected this dataset

The earlier ULB dataset contains real transactions, but its important inputs are anonymous PCA components called V1–V28. We cannot reconstruct those components from a normal merchant and amount form because the original transformation is unavailable.

PaySim contains sender and receiver balances, but it models mobile-money transfers. A credit-card purchase usually does not give the cardholder access to the merchant's bank

balance.

Sparkov provides card purchases with understandable fields and histories. That makes it suitable for demonstrating a complete interactive credit-card ML system. The Fraud Detection Handbook simulator and IBM TabFormer are other credible synthetic alternatives; switching again was unnecessary once this source and its feature code were verified.

4. Training and prediction are different

Training happens before deployment. We give an algorithm many examples with known labels. It adjusts its internal parameters to reduce prediction errors. We then save the fitted preprocessing and model.

Prediction happens when you click Analyze. The app prepares your entered details in exactly the same way as training, loads the already fitted model, and calculates a score. It does not search for an identical saved transaction and does not train again on your input.

The saved file contains learned numerical parameters. It is not a list of hardcoded answers.

```
BEFORE DEPLOYMENT
Source records -> earlier-history features -> chronological split
-> fit LR and XGBoost -> choose on validation -> evaluate later data
-> save preprocessing, model, and measured results

WHEN USING THE WEBSITE
Your payment details -> same feature definitions
-> saved preprocessing -> selected XGBoost model
-> score and cutoff -> predicted class and explanation
```

5. What the form asks and why

Form field	Meaning and use
Payment amount (USD)	The value of the proposed purchase in US dollars.
Purchase category	The type of purchase, such as groceries, travel, or online shopping.
Merchant	The shop or business; used for familiarity checks when actual history is supplied. The name itself is not a predictive number.
Payment date	Supplies the day of the week.
Payment time	Supplies the clock time. This is not how long the payment takes to process.
Average earlier payment (USD)	Average value of this card's previous purchases.
Minutes since previous payment	Gap between the latest earlier purchase and the new payment.
Payments in previous hour	Number of earlier payments within 60 minutes.
Payments in previous 24 hours	Number within a day; includes those in the previous hour.
Used this merchant before?	Whether the card has previously paid this merchant.

Form field	Meaning and use
Used this purchase category before?	Whether the card has previously made this kind of purchase.

All history describes activity **before** the payment being checked. You can enter a hypothetical scenario for a presentation, but a hypothetical transaction has no confirmed answer label.

The form rejects contradictory details. For example, “last payment 10 minutes ago” cannot go with “zero payments in the previous hour.” A payment amount must be positive. Rupees must not be entered as though they were dollars.

6. What feature engineering means

Feature engineering turns the available facts into useful model inputs.

For example, a $300 purchase means something different when the earlier average is $30 rather than $500. The model receives both the amount and the ratio:

```
Amount compared with earlier average = current amount / earlier average
300 / 30 = 10 times the earlier average
```

The model also receives recent payment counts and merchant familiarity. The customer identifier is only used to group that customer's earlier purchases; it is not a direct classifier input.

Clock time uses two numerical coordinates, sine and cosine. You can explain this as putting time around a clock face. It prevents the model from treating 23:59 and 00:01 as far apart. Users do not enter these coordinates.

There are 13 internal inputs because the time-of-day representation uses two numbers. Readable names are used in the interface and SHAP plots.

7. Three ways to supply earlier activity

Enter activity details: you provide the average, counts, time gap, and familiarity. The app derives the exact trained feature definitions from those summaries.

Use demo card history: select a synthetic customer alias. The app automatically calculates history using only transactions strictly before your chosen payment time. The demo data comes from 2020; choose a date in that period for a historical replay.

No earlier history: the app marks context as unavailable. It does not invent an average or recent purchases. Training medians fill undefined numeric values inside the saved pipeline, while a separate availability flag tells the model that history is missing.

To prepare this fallback, 20% of training rows had their history masked independently of their fraud labels. The fallback has its own validation-selected cutoff and separately reported results. Those results measure withheld history on later simulated payments, not actual new-bank customers.

8. What preprocessing does

A model expects consistent numerical inputs. Preprocessing makes that happen.

• **Cleaning:** check dates, numeric amounts, target labels, missing required values, and duplicates.

• **Missing-value handling:** unavailable numerical history uses medians calculated from training data.

• **One-hot encoding:** a category becomes a set of category indicator columns. The user picks “Online shopping”; the saved encoder creates the matching numerical representation.

• **Scaling:** Logistic Regression places numeric fields on comparable scales. XGBoost does not need this scaling step.

• **Feature order:** the app supplies the exact named columns used during training.

All learned preprocessing is fitted on training data only and saved with the classifier as a pipeline. This prevents the website from using a different scaler or encoder from the one used to train the model.

9. Logistic Regression in plain English

Logistic Regression is our baseline classifier. It learns a weight for each processed feature, adds the weighted effects, and converts the total into a score between zero and one.

A positive effect can increase the fraud score; a negative effect can decrease it. This is a relatively simple way to model the relationship between inputs and the label.

It is useful as a baseline because we can check whether the more advanced method actually improves detection. In this experiment it performed substantially worse than XGBoost; that is a result to explain honestly, not hide.

10. XGBoost in plain English

A decision tree asks a sequence of questions about features. For example, a tree might split on amount and then on purchase category. The actual questions and cutoffs are learned from training records.

XGBoost combines many small trees. Each new tree helps correct errors remaining in the current model. Their contributions add together to produce the final score. That process is called gradient boosting.

Our selected candidate uses 350 trees with a maximum depth of 5. The app uses this one fitted XGBoost pipeline for predictions. Logistic Regression is a comparison model, not a second opinion secretly combined with the output.

Random Forest, used in older experiments, averages the outputs of varied trees trained in parallel. XGBoost builds its trees sequentially. Random Forest is not part of the finished app's active comparison.

11. Why there are three data periods

Training: 333,495 earlier transactions teach the models their parameters.

Validation: 114,518 subsequent transactions select model settings and alert cutoffs.

Test: 107,706 later transactions measure the chosen model's errors.

The split uses time because the intended task is to score a later transaction using earlier information. Transactions at the same timestamp do not enter each other's history. Future transactions and fraud answer labels are excluded from historical features.

The tuning search compares two Logistic Regression regularization settings and two XGBoost depths. It selects by validation F1, with average precision as a tie-breaker. It does not select the winner by the final test table.

Earlier project experiments already examined aggregate results from this public source's later period. Therefore, call this a development holdout, not a completely untouched external evaluation.

12. Why accuracy can mislead

The test set has 107,706 payments, but only 176 are labeled fraud. A program that always says legitimate would already be correct about 99.837% of the time while finding no fraud.

Our XGBoost accuracy is about 99.896%. The more useful facts are that it caught 119 fraud cases, missed 57, and raised 55 false alarms.

Metric	Plain-English question
Precision	When the model flags a payment, how often is the flag correct?
Recall	Of all recorded fraud cases, how many did the model catch?
F1	How well does the model balance precision and recall?
PR-AUC	Across possible cutoffs, how well does it balance finding rare fraud with making correct flags?
ROC-AUC	Across cutoffs, how well do scores rank fraud above legitimate payments?
Confusion matrix	How many correct decisions, false alarms, and missed fraud cases occurred?
Threshold	The score cutoff at which a payment is classified as suspected fraud.
Class imbalance	Fraud is much rarer than legitimate activity.

The selected model has 68.39% precision, 67.61% recall, 68.00% F1, 71.95% PR-AUC, and 99.58% ROC-AUC with earlier history. A high ROC-AUC does not cancel the actual missed-fraud count at the selected cutoff.

Without earlier history, the measured precision is 58.16%, recall 64.77%, and F1 61.29%. That is why the fallback's limitations are visible.

13. Understanding the result card

“Predicted fraudulent” means the score reached the applicable cutoff. “Predicted legitimate” means it did not. Neither establishes what actually happened.

The score is uncalibrated. A score of 90 out of 100 is not automatically a verified 90% real-world chance of fraud.

Risk bands are a review policy: High starts at the cutoff, Medium spans half the cutoff to the cutoff, and Low is below half the cutoff. History-available and no-history contexts have different validation-selected cutoffs.

The app does not know a person's intention or whether a conversation is a scam. It analyzes the supplied transaction features.

14. What SHAP explains

SHAP assigns a contribution to each actual model input, relative to a reference based on earlier training rows. Contributions can raise or lower the score.

A local waterfall starts from the reference and adds those contributions. In this app, the contributions use the model's internal log-odds scale. Converting the final total with the logistic function recovers the model score. The code verifies that the explanation reconstructs the prediction.

A global chart averages absolute contributions over 128 randomly sampled later test records. It tells you which features strongly influenced that sample's predictions. It is not an evaluation on only 128 records; model performance uses all 107,706 later transactions.

“Amount raised the model score” is a statement about the model's calculation. It does not prove that the amount caused fraud.

15. Walkthrough of every page

Fraud prediction: enter your own payment details and context, analyze, and explain the result. Change a value to demonstrate actual inference. Old predictions clear on an edit.

Overview: distinguish source statistics from session activity. The dataset contains recorded outcomes; session counts contain model predictions. Point out that USD amount statistics describe purchases in the source, not money processed by the app.

Batch analysis: download and upload the template. The app validates its columns, computes strictly earlier activity, scores every row, and offers a full download. Each customer's first transaction in that file starts without history. The app does not claim to read every bank's PDF statement.

Model performance: explain the two-model comparison and the 119 caught / 57 missed / 55 false-alarm counts. Switch to the missing-history context to show its separate evaluation.

Explainable AI: explain the latest manual payment or a recorded test example. The sample cases deliberately include correct predictions and mistakes. Labels shown for recorded examples never enter the classifier.

Transaction monitoring: search session analyses and filter by risk. It is a session log, not an actual bank feed. Refreshing or restarting can clear it. It retains up to 5,000 unique transaction IDs.

16. A five-minute demonstration

1. Introduce the topic and disclose Sparkov's synthetic source.

2. Open Fraud prediction. Enter a $100 online-shopping payment with an earlier average of $75, a 120-minute gap, zero payments in the last hour, and three in the last day.

3. Click Analyze transaction. Explain the actual class and score that appear; do not promise a particular answer in advance.

4. Change the amount, analyze again, and show that a real model recalculates the score. A score can change without crossing the classification cutoff.

5. Open Model performance. Describe caught fraud, missed fraud, false alarms, and the baseline comparison.

6. Open Explainable AI and explain one actual contribution.

7. Upload the provided CSV template, download its genuine predictions, and show the resulting session log.

8. End with the limits: synthetic source, imperfect detection, uncalibrated scores, and no live bank connection.

17. Ten likely viva questions and answers

1. What problem does your project solve?

It demonstrates supervised classification of potentially fraudulent credit-card purchases, with an interactive interface, batch processing, and measured evaluation.

2. Is your dataset real?

No. Sparkov is synthetic card-transaction data. The training is genuine, but the reported performance is on simulated transactions.

3. Why did you replace the PCA-based interface?

Ordinary users cannot provide V1–V28, and the original PCA transformation is unavailable. Sparkov supports readable fields that match the manual form.

4. How does the model predict fraud?

The app derives the trained features, applies saved preprocessing, runs the selected XGBoost model, and compares its score with the validation-selected cutoff.

5. Why compare Logistic Regression and XGBoost?

Logistic Regression provides a simple baseline. XGBoost can learn interactions and nonlinear patterns. Its better validation F1 selected it in this experiment.

6. How did you avoid leakage?

Preprocessing is learned on training data only. Model settings and cutoffs use validation. Historical features exclude future transactions, same-time transactions, and fraud answer labels.

7. Why not present accuracy alone?

Fraud is rare. Always predicting legitimate gets very high accuracy but catches no fraud. Precision, recall, F1, and error counts reveal the practical mistakes.

8. What happens when a customer has no history?

The model has explicit missing-history handling learned using masked training examples, training-only imputation, and a separately validated cutoff. Its weaker performance is reported separately.

9. Are the explanations actual model explanations?

Yes. SHAP contributions are computed from the selected model and reference training rows. The code checks that they reconstruct the score. They are not causal proof.

10. Can this immediately replace a bank's fraud system?

No. That would require authorized real data, independent evaluation, calibrated scores if probabilities are needed, secure integration, and monitoring. This project demonstrates the ML and application workflow with its limitations stated.

18. Which files should you study first?

Read these in order:

1. `app.py`: starts the dashboard.

2. `shield_dashboard.py`: collects inputs and displays predictions, charts, and explanations.

3. `manual_card_core.py`: validates entered summaries and derives features.

4. `card_core.py`: computes strictly earlier spending history.

5. `shield_core.py`: loads the saved model, validates CSVs, predicts, and computes SHAP.

6. `train_shield.py`: cleans the source, splits time periods, fits/tunes models, evaluates, and exports.

7. `shield_artifacts/metadata.json`: records the actual data, model settings, cutoffs, and measured results.

You do not have to memorize every line. Be able to trace one payment from its form fields to its derived features, through the saved pipeline, to its score and cutoff.